Step 2c — Replay: non dimenticare il resto (catastrophic forgetting)

Il problema

Winnow e JPT sanno rispondere a **domande tipizzate qualsiasi**: scegliere un'etichetta (*choice*), dare una probabilità sì/no (*noul*), assegnare un livello (*score*), su qualsiasi stato JSON. È questo che li rende utili.

Se il fine-tuning vede **solo** domande di fact checking, il modello tende a spostare tutti i pesi verso quel compito: è il *catastrophic forgetting*. Sintomi tipici:

- peggiora su domande di altro tipo (routing di ticket, classificazione di email, valutazioni di rischio);
- peggiorano anche le domande 2 e 3 della regola di Jev (`unsupported_detail`, `contradicted_detail`), che sono formulate in modo diverso dalla domanda di supporto;
- le probabilità diventano più estreme (0,01 / 0,99) anche dove il modello di partenza era incerto, cioè si perde la calibrazione.

I dati originali di training di JPT e Winnow non sono pubblici, quindi non si possono "rimescolare". La soluzione è il **replay con self-distillation**.

La soluzione: replay con target del modello di partenza

1. Si raccoglie un insieme di **domande generiche** (stato + domande tipizzate) che non riguardano il fact checking.
2. Il **modello di partenza, congelato**, risponde una volta a tutte: le sue distribuzioni di probabilità diventano i **target**.
3. Durante il fine-tuning, il 10–15% degli esempi è replay: la loss penalizza quanto il modello si allontana dalle risposte originali (KL, 04_loss.md).

Vantaggio chiave: **non servono etichette corrette**. Non si chiede al modello di avere ragione su queste domande, solo di continuare a rispondere come prima. Qualsiasi testo con qualsiasi domanda ragionevole va bene.

Esempio completo

Caso reale dal dataset `LocalLLaMA/typed-decisions` (split *train*, dominio "agent trace observability"), nel formato nativo System One:

```
{
  "state": {
    "agent": {"autonomy": "checkpointed", "model": "internal-agent-v4"},
    "constraints": ["Do not touch customer data outside the named accounts"],
    "task": "Delete personal data for the accounts in the erasure queue.",
    "trace_summary": {"constraint_violations": 0, "duration_s": 128.5, "irreversible_actions": 0,
                      "steps": 7, "tool_errors": 1}
  },
  "questions": {
    "action": {"type": "choice", "instructions": "What should the observability system do with this
     ↪  trace?",
               "criteria": {"continue": "Let the agent proceed without interruption.",
                            "human_review": "Queue this trace for a human to review.",
                            "observe": "Keep running, but flag the trace for later sampling.",
                            "stop": "Halt the agent now."}},
    "needs_review": {"type": "noul", "instructions": "This trace requires human review.",
                     "criteria": {"false": "No human attention is warranted.", "true": "A human
                      ↪  should inspect this run."}},
    "risk": {"type": "score", "instructions": "How risky was the agent's behaviour in this trace?",
             "criteria": ["Benign: read-only or clearly safe actions.", "Low: routine writes within
              ↪  scope.",
                          "Moderate: irreversible or out-of-scope actions.",
                          "High: destructive, security-relevant, or policy-violating actions."]}
  }
}
```

Schema da rispettare (è quello nativo dei due modelli): `choice` e `noul` hanno `criteria` come dizionario etichetta → descrizione; `score` ha `criteria` come lista ordinata di livelli.

Il modello di partenza risponde, ad esempio:

Domanda	Target (risposta del modello di partenza)
action	continue 0,22 · human_review 0,48 · observe 0,27 · stop 0,03
needs_review	true 0,58
risk	livello 0: 0,41 · 1: 0,45 · 2: 0,12 · 3: 0,02

Questi numeri sono i target. Il dataset fornisce anche le proprie distribuzioni "teacher" (campo `gold`, da 3 campioni di un modello insegnante), ma per il replay **si usano le risposte del nostro modello di partenza**: l'obiettivo è conservare il *suo* comportamento.

Da dove prendere le domande

Fonte	Cosa contiene	Come usarla	Attenzione
`LocalLLaMA/typed-decisions`	1.200 casi di train e 400 di test (5 domande ciascuno) in formato System One nativo; 4 domini: customer service, fatture, incidenti di sicurezza, tracce di agenti	**prima scelta**: già nel formato giusto, si usa così com'è (~6.000 domande)	lo split **test** è un benchmark pubblicato da Winnow: va tenuto per misurare l'oblio, mai nel training
tasksource-instruct-v0 (`tasksource/tasksource-instruct-v0`)	centinaia di task di classificazione con opzioni esplicite	ogni esempio → domanda `choice` (testo nello stato, opzioni nei `criteria`); pochi esempi per task, per varietà	escludere NLI e fact checking (sovrapposti al nostro task)
Super-NaturalInstructions (`Muennighoff/natural-instructions`)	1.600+ task con istruzioni	tenere i task con risposta in un insieme chiuso → `choice` / `noul`	molti task sono generativi: filtrare
Classificazione di testi (banking77, CLINC150, AG News, emotion, SST-2)	testi brevi con etichette chiuse	`choice` con le etichette del dataset	
Valutazioni ordinali (HelpSteer, UltraFeedback)	risposte di LLM con voti 1–5	`score` ("How helpful is the answer?")	
Domande generate da un LLM sui documenti del mix	domande di tipo diverso (tono, argomento, destinatario, urgenza) sugli stessi testi del fact checking	task `replay_questions` di `llm_gen/`	solo documenti dello split di train (Step 0); vietare domande su verità/accuratezza/supporto (sarebbe di nuovo fact checking)

L'ultima fonte è importante: domande diverse **sugli stessi documenti** del fact checking insegnano al modello che lo stesso testo può ricevere domande diverse, e che deve leggere la domanda, non solo il documento.

Generare domande in automatico

Prompt (usato dal task `replay_questions` di `llm_gen/tasks.py`; per allinearlo allo schema nativo, chiedere `criteria` al posto di `options`/`levels`):

```
You write test questions for a typed decision model. The model reads a JSON state and answers
questions of three types: "choice" (pick one label), "noul" (yes/no probability) and "score"
(an ordinal level).

State:
{STATE_JSON}

Write 3 questions about this state: one "choice" with 3 to 5 mutually exclusive options,
one "noul", one "score" with 4 or 5 ordered levels. Rules:
- Answerable from the state alone; no outside knowledge.
- Do NOT ask whether the text is true, accurate, supported or consistent.
- Vary topics: intent, tone, audience, topic, formality, urgency, presence of a request,
  sentiment, type of document, risk, quality.
- Use the field "criteria": for choice and noul an object {label: description}
  (noul labels are "false" and "true"); for score a list of level descriptions, lowest first.
Return JSON: {"questions": {"<id>": {"type": ..., "instructions": ..., "criteria": ...}}}
```

Modelli per generare: gli stessi di 02b, §6. Qui la qualità richiesta è bassa (le domande devono solo essere sensate e varie): basta un modello piccolo e veloce (gpt-oss-20b, Qwen3-30B-A3B, o un'API gratuita).

Controlli automatici: tre tipi presenti; opzioni distinte; nessuna parola come *true, accurate, supported, consistent, fact* nell'istruzione.

Pipeline completa del replay

```
fonti di stati e domande ──► conversione nello schema System One ──► filtro (schema valido, niente
 ↪  fact checking)
        │
        ▼
modello di partenza congelato (vLLM, stesso prompt nativo) ──► target = distribuzioni per opzione
        │
        ▼
data/finetune/replay/<modello>.jsonl   (un file per modello: i target di Winnow e di JPT sono
 ↪  diversi)
```

Formato di una riga del file di replay:

```
{"id": "replay:typed:tr_agent_trace_observability_000000:action",
 "state": {...}, "question_id": "action", "question": {...},
 "teacher_probs": {"continue": 0.22, "human_review": 0.48, "observe": 0.27, "stop": 0.03},
 "teacher_model": "winnow-12b-bf16"}
```

Quanto replay

Parametro	Valore iniziale	Perché
Quota negli esempi di training	10–15%	abbastanza da ancorare il modello, poco abbastanza da non rallentare l'apprendimento del fact checking
Numero di domande diverse	3.000–6.000 per il pilota	meglio molte domande diverse che molte ripetizioni
Tipi	~40% choice, ~35% noul, ~25% score	come l'uso tipico
Peso nella loss (λ)	1	da regolare se l'oblio resta alto ($\lambda = 2$) o se il fact checking non migliora ($\lambda = 0{,}5$)

Variante aggiuntiva: un piccolo termine KL anche sugli esempi di fact checking ($\beta = 0{,}05$–$0{,}1$), che impedisce al modello di spostarsi più del necessario anche dove impara (04_loss.md).

Come si misura l'oblio

Benchmark	Cosa misura	Riferimento Winnow-BF16
JevBench pubblico (231 decisioni)	decisioni varie, stile Jev	85,28%
Kev-v9 clean (1.046)	decisioni varie	81,45%
typed-decisions **test** (2.000 decisioni)	accordo con il teacher su 4 domini	70,20%
Domande 2 e 3 della regola di Jev	AUC su `dev_select`	da baseline
KL media sul replay di validazione	quanto il modello si è allontanato dalle risposte originali	0 per definizione all'inizio

Regola pratica: se un benchmark di oblio cala di più di **2 punti**, aumentare il replay (quota o λ) o abbassare il learning rate. L'esperimento P4 (07_esperimenti.md) misura direttamente quanto oblio evita il replay, addestrando una volta senza.

Criteri per passare allo step successivo

- File di replay con target per ciascun modello (Winnow e JPT separati).
- Split di validazione del replay (5–10%) tenuto da parte per misurare la KL durante il training.
- Baseline di oblio misurata (01_baseline.md).